Event Map App

Сервис поиска и публикации локальных мероприятий

СРС №1 · Архитектура, API и безопасность

Дисциплина: Разработка веб-приложений

Студент: Фамилия Имя, группа ____

Преподаватель: Самат Илияс Мырзабекулы

Алматы, 2026

MVP · Python 3.12 · Flask / Django · JWT

Предметная область и бизнес-задача

Локальные события — на карте, с фильтрами и понятной регистрацией

- **Пользователь** — находит концерты, лекции, выставки и встречи рядом с собой
- **Организатор** — публикует мероприятие и видит отклик аудитории
- **Ключевая ценность** — геолокация, категория и дата в одном поиске
- **Аудитория** — жители города и небольшие организаторы
- **MVP** — три сервиса, REST + JWT, единые ошибки и контракт OpenAPI

Сценарий MVP: найти событие → опубликовать → отметить участие → увидеть статистику

Пользовательские истории

Основные истории

- **US-1** — карта: маркеры событий в радиусе 10 км
- **US-2** — фильтр по категории и дате
- **US-3** — публикация события организатором
- **US-4** — отметка «пойду» → счётчик +1
- **US-5** — статистика «пойду» / «интересно»
- **US-6** — регистрация и выбор роли

Definition of Done

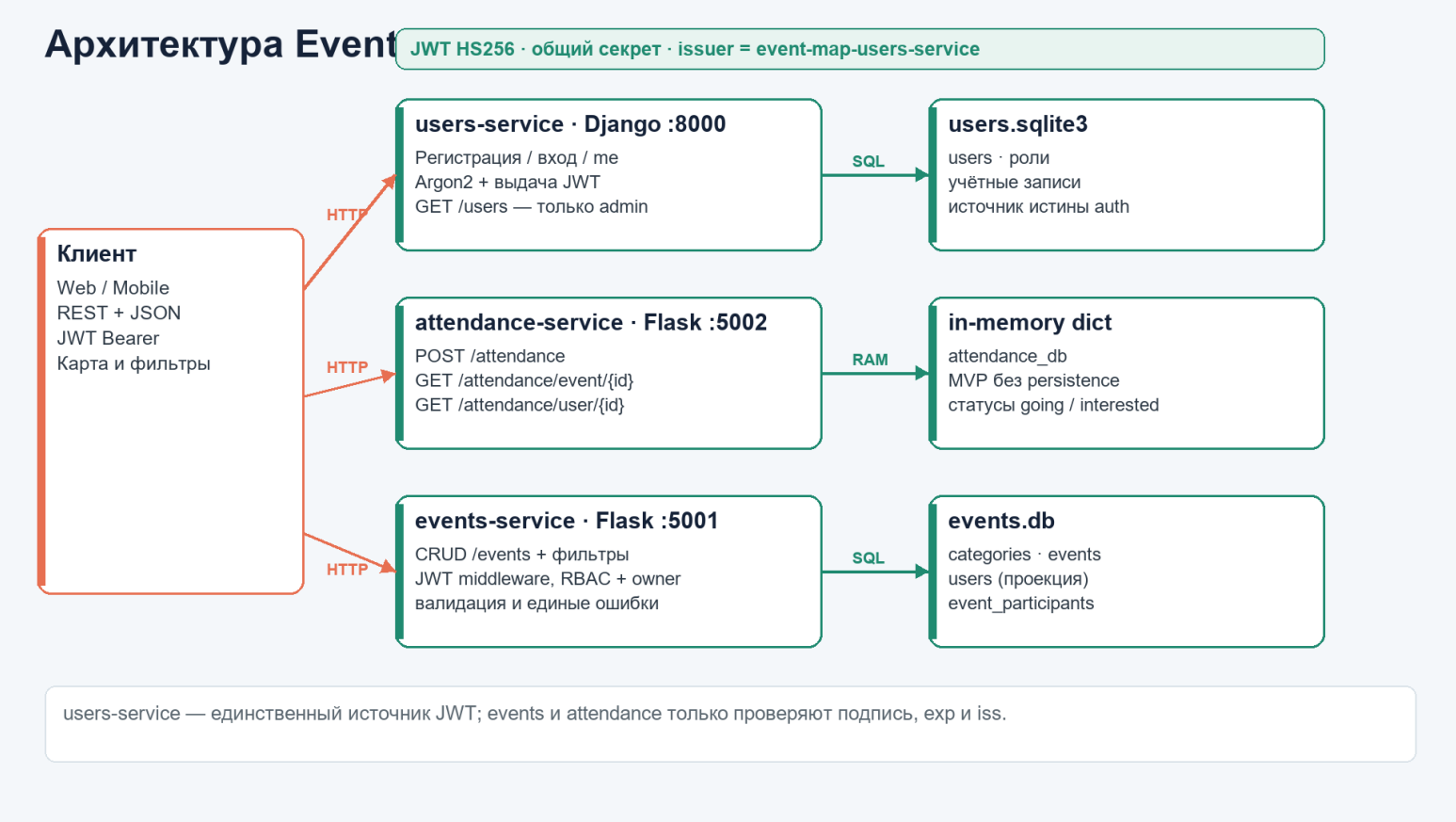
- US-3: organizer заполняет форму → событие видно всем
- US-4: отметка сохраняется, счётчик обновляется
- US-6: user или organizer; admin — отдельно
- **Критерий** — 401/403/400 возвращаются в едином формате
- **Проверка** — ручной чек-лист + автотесты сервисов

Выбор технологий

Критерий	Flask	Django	FastAPI
ORM и миграции	нет из коробки	ORM + миграции	SQLAlchemy / Alembic
Auth и admin	собрать вручную	готовые auth + admin	собрать вручную
Скорость MVP	высокая	средняя	высокая
Async	нет	нет	да
Решение	events, attendance	users-service	не выбран для MVP

Единый стек Python 3.12; SQLite локально, PostgreSQL через DATABASE_URL.

Архитектура приложения



- Клиент → REST + JWT Bearer → три сервиса
- users-service — источник токенов и ролей
- events-service — доменная логика и events.db
- attendance-service — отметки участия, in-memory MVP

Структура репозитория и конфигурация

Структура

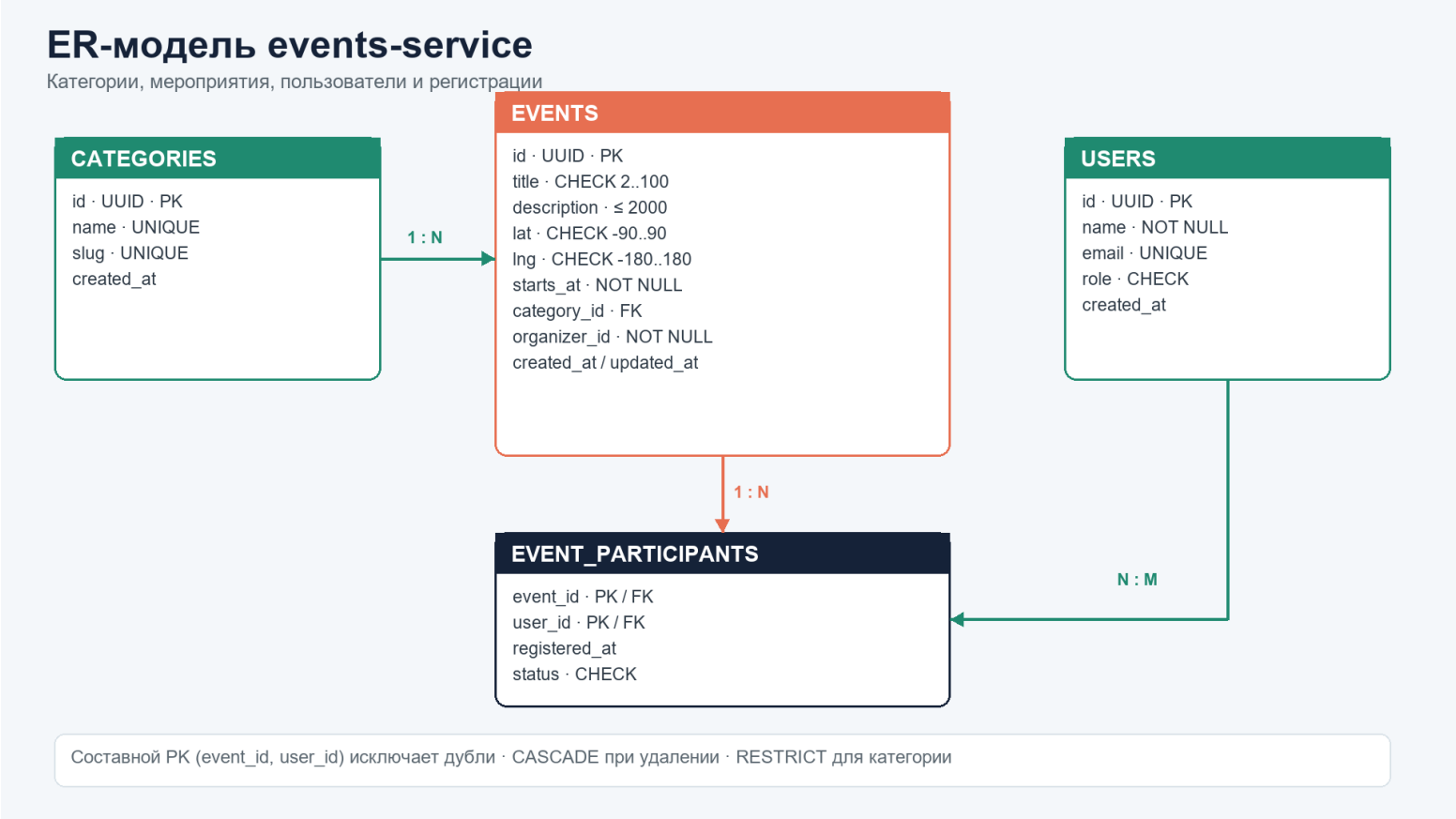
- **users-service/** — Django: auth, JWT, admin
- **events-service/** — Flask: events, Alembic, OpenAPI
- **attendance-service/** — Flask: отметки участия
- **docs/** — требования, auth, security, CPC1
- **README.md** — запуск, тесты, PR1–PR4

Переменные окружения

- **JWT_SECRET** — общий секрет подписи
- **JWT_ISSUER** — event-map-users-service
- **JWT_EXPIRES_MINUTES** — 60
- **DJANGO_SECRET_KEY / DJANGO_DEBUG** — секрет / 0
- **DATABASE_URL** — SQLite или PostgreSQL
- **FLASK_DEBUG** — 0

Секреты не хранятся в коде; .env.example содержит только шаблон.

ER-модель и связи



- categories 1:N events — RESTRICT
- users N:M events через event_participants — CASCADE
- Составной PK (event_id, user_id) исключает дубли
- users в events-service — проекция для FK и регистраций

Миграции, ограничения и индексы

Схема создаётся только Alembic; ограничения проверяются на уровне БД

- **0001** — categories и events — базовые таблицы
- **0002** — categories.slug + составной индекс
- **0003** — users и event_participants — связь N:M
- **CHECK** — title 2..100, description ≤ 2000, lat/lng, role/status
- **Индексы** — ix_events_category_id, ix_events_category_starts_at, ix_event_participants_user_id
- **Транзакции** — commit/rollback в database.db_session; verify_tx.py

Проверка: alembic upgrade head → downgrade -1 → upgrade head.

Контракт REST API

Метод и путь	Доступ	Назначение
POST /auth/register	public	регистрация user / organizer
POST /auth/login	public	проверка пароля и выдача JWT
GET /auth/me	auth	текущий пользователь
GET /users	admin	список пользователей
GET /events	public	список + фильтры и пагинация
POST /events	organizer / admin	создание события
PUT/PATCH/DELETE /events/{id}	owner / admin	изменение или удаление
POST /attendance	auth	отметка участия
GET /attendance/event/{id}	public	статистика события
GET /attendance/user/{id}	self / admin	отметки пользователя

OpenAPI 3.0: events-service/openapi.yaml · версия 4.0.0 · BearerAuth.

Валидация и единый формат ошибок

Правила валидации

- **title** — 2..100 символов
- **description** — не более 2000
- **lat / lng** — -90..90 / -180..180
- **starts_at** — ISO 8601
- **category** — значение из справочника
- **limit / page** — 1..100 / ≥ 1

Пример ошибки

- HTTP/1.1 400 BAD REQUEST
- {
- "error": {
- "code": "ValidationError",
- "message": "Ошибка валидации",
- "details": [{"field": "lat", "message": "..."}]
- }
- }

Один формат для users, events и attendance; stack trace не возвращается.

Пагинация, фильтрация и пример ответа

Запрос

- GET /events
- ?category=concert
- &date_from=2026-10-01
- &date_to=2026-10-31
- &limit=2&page=1

Ответ 200 OK

- {
- "items": [
- {"id": "...", "title": "Концерт в парке",
- "category": "concert",
- "starts_at": "2026-10-01T19:00:00Z"}
-],
- "page": 1, "limit": 2, "total": 1
- }

Фильтр по категории и датам, сортировка по starts_at, limit ≤ 100.

Аутентификация: Argon2 + JWT

Поток входа

- **1** — POST /auth/register → Argon2-хеш пароля
- **2** — POST /auth/login → проверка Argon2
- **3** — users-service выдаёт JWT HS256
- **4** — клиент передаёт Authorization: Bearer <token>
- **5** — сервис проверяет подпись, exp, iss
- **6** — sub и role → g.current_user

Claims

- {
- "sub": "user-uuid",
- "role": "organizer",
- "iss": "event-map-users-service",
- "exp": 1760000000,
- "iat": 1759996400
- }

TTL 60 минут; пароль не хранится в открытом виде; ошибка входа не раскрывает пользователя.

Авторизация и матрица ролей

Операция	user	organizer	admin
GET /events	да	да	да
POST /events	нет	да	да
PUT/PATCH/DELETE своего события	нет	да	да
PUT/PATCH/DELETE любого события	нет	нет	да
POST /attendance	да	да	да
GET /users	нет	нет	да
GET /attendance/user/{id}	self	self	любой

401 — нет или просрочен токен; 403 — роль или объект запрещён. Проверки: require_role + _authorize_event.

Угрозы API и меры защиты (OWASP)

Угроза	Мера	Проверка
API1 BOLA	owner-check + admin override	чужое событие → 403
API2 Broken Auth	Argon2, JWT exp/iss, 401	неверный пароль → 401
API3 Mass Assignment	id и role только из JWT	подмена organizer_id игнорируется
API4 Resource Consumption	limit 1..100, длины полей	limit=1000 → 400
API5 BFLA	require_role, admin-only /users	user → /users 403
API8 Misconfiguration	секреты в .env, DEBUG=0	500 без stack trace

Подробнее: docs/security.md. Rate limiting и refresh-токены — план до СРС №2.

Демонстрация работы API

POST /events → 201 Created

```
$ POST /events  ·  Authorization: Bearer <ORG_TOKEN>

HTTP/1.1 201 CREATED
Content-Type: application/json

{
  "id": "0f7c2a9e-4b1d-...",
  "title": "Концерт в парке",
  "organizer_id": "b3e1a7c2-...",
  "category": "concert",
  "starts_at": "2026-10-01T19:00:00Z"
}

organizer_id взят из проверенного JWT
```

201 Created — создание события организатором

POST /events → 400 Bad Request

```
$ POST /events  ·  Authorization: Bearer <ORG_TOKEN>

HTTP/1.1 400 BAD REQUEST

{
  "error": {
    "code": "ValidationError",
    "message": "Ошибка валидации",
    "details": [
      {"field": "lat", "message": "должно быть числом"},
      {"field": "lng", "message": "диапазон -180..180"}
    ]
  }
}
```

400 Bad Request — ошибка валидации

DELETE /events/{id} → 403 Forbidden

```
$ DELETE /events/<ID>  ·  Bearer <USER_TOKEN>

HTTP/1.1 403 FORBIDDEN

{
  "error": {
    "code": "Forbidden",
    "message": "Недостаточно прав",
    "details": [
      {"field": "role",
       "message": "требуется organizer или admin"}
    ]
  }
}
```

403 Forbidden — роль user не может удалить

DELETE /events/{id} → 204 No Content

```
$ DELETE /events/<ID>  ·  Bearer <ADMIN_TOKEN>

HTTP/1.1 204 NO CONTENT
Date: Fri, 23 Oct 2026 12:00:00 GMT

<тело ответа отсутствует>

admin: роль + owner-check пройдены
```

204 No Content — admin удаляет событие

Фрагменты реальных HTTP-ответов; тесты: users 10, events 31, attendance 10 — все проходят.

Статус проекта и план до СРС №2

Сделано: PR1–PR4

- **PR1** — бизнес-задача, US, архитектура, /health
- **PR2** — OpenAPI, примеры, пагинация, ошибки
- **PR3** — ER, миграции, индексы, N:M, транзакции
- **PR4** — JWT, Argon2, роли, owner-check
- **Тесты** — 10 + 31 + 10, все проходят

План и риски

- **PR5** — attendance в БД, синхронизация users
- **PR6** — rate limiting, refresh-токены, HTTPS
- **PR7** — docker-compose, CI, финальная защита
- **Риск** — JWT без отзыва; attendance in-memory
- **Риск** — users-проекция не синхронизируется автоматически

Репозиторий: github.com/banumm/event-map-app · ветка main · коммит: <вставить SHA>

Заключение

- MVP покрывает ключевой сценарий: найти событие → опубликовать → отметить участие.
- Контракт зафиксирован в OpenAPI; ошибки и доступ проверяются тестами.
- Безопасность: Argon2, JWT, RBAC + owner-check, анализ OWASP API Top 10.
- Дальше: persistence для attendance, rate limiting, CI/CD и production-конфигурация.

Вопросы?

github.com/banumm/event-map-app · СРС №1 · 2026